

PrepME AI — Day 1 Learning Notes

1. Full Stack Architecture

A modern full-stack application is generally divided into 3 major parts:

Frontend

The UI users interact with.

Examples:

- buttons
- forms
- dashboards
- pages

Technology used:

- React

Frontend runs in browser.

Backend

Handles logic and APIs.

Examples:

- login validation
- authentication
- database operations
- business logic

Technology used:

- Node.js
- Express.js

Backend runs on server.

Database

Stores application data.

Examples:

- users
- passwords
- progress
- notes

Technology planned:

- MongoDB
-

2. Project Structure

```
PrepME-AI
|
├─ client  → frontend
├─ server  → backend
```

client

Contains React frontend code.

server

Contains Node.js backend code.

3. React Basics

React is a frontend library created by Meta.

Purpose:

- build reusable UI
 - manage dynamic data
 - update screen efficiently
-

4. Vite

Vite is a frontend development/build tool.

Purpose:

- fast React setup
- local development server
- fast refresh

Command used:

```
npx create-vite@latest client
```

5. npm and Packages

npm = Node Package Manager.

Purpose:

- install external libraries/packages

Example:

```
npm install axios
```

A package is reusable code written by other developers.

Examples:

- React
 - Axios
 - Express
 - CORS
-

6. node_modules

Contains downloaded project dependencies.

Generated automatically after:

```
npm install
```

Never manually edit this folder.

7. package.json

Very important file.

Contains:

- project information
- dependencies
- scripts

Example:

```
"dependencies": {  
  "react": "..."  
}
```

8. React Components

A React component is a JavaScript function that returns UI.

Example:

```
function App() {  
  return <h1>Hello</h1>  
}
```

Components are reusable.

Examples:

- Navbar
- Sidebar
- UserCard

- Button
-

9. JSX

JSX = HTML-like syntax inside JavaScript.

Example:

```
<h1>PrepME AI</h1>
```

React converts JSX into JavaScript internally.

10. Props

Props are used to pass data from one component to another.

Example:

```
<UserCard name="Shweta" />
```

Inside component:

```
props.name
```

Props make components reusable.

11. State

State stores dynamic data inside a component.

Created using:

```
useState()
```

Example:

```
const [count, setCount] = useState(0)
```

When state changes:

- React re-renders component
 - UI updates automatically
-

12. Event Handling

React handles user interactions using events.

Example:

```
onClick={handleClick}
```

Used for:

- button clicks
 - form submissions
 - typing inputs
-

13. Controlled Components

Inputs controlled by React state.

Example:

```
<input  
  value={email}  
  onChange={(e) => setEmail(e.target.value)}  
>
```

Flow:

- user types
 - React captures value
 - state updates
 - UI re-renders
-

14. Forms in React

Forms collect user data.

Example:

- login form
- signup form

Form submission:

```
<form onSubmit={handleSubmit}>
```

15. preventDefault()

Used to stop default browser behavior.

Example:

```
event.preventDefault()
```

Without this:

- form reloads page

Important for Single Page Applications (SPA).

16. useEffect

Used for side effects.

Most commonly:

- API calls
- fetching data

Example:

```
useEffect(() => {  
  
}, [])
```

Runs when component loads.

17. Backend Basics

Backend built using:

- Node.js
- Express.js

Setup commands:

```
mkdir server  
cd server  
npm init -y  
npm install express
```

18. Express.js

Express is a backend framework for Node.js.

Purpose:

- create APIs
 - handle routes
 - process requests
-

19. Express Server

Basic server setup:


```
const express = require("express")

const app = express()
```

20. Routes

Routes define backend endpoints.

Example:

```
app.get("/", (req, res) => {
  res.send("Backend running")
})
```

Examples of routes:

- /login
- /signup
- /users

21. Request and Response

Backend receives request:

```
req
```

Backend sends response:

```
res
```

22. APIs

API = communication bridge between frontend and backend.

Frontend sends request. Backend processes logic. Backend sends response.

23. HTTP Methods

GET

Fetch data.

Example:

```
GET /users
```

POST

Send/create data.

Example:

```
POST /login
```

24. JSON

JSON is the standard format for frontend-backend communication.

Example:

```
{
  "email": "abc@gmail.com",
  "password": "123456"
}
```

25. express.json()

Middleware used to parse JSON request body.

Example:

```
app.use(express.json())
```

Without this:

```
req.body
```

will not work correctly.

26. Axios

Axios is used for API requests from frontend.

Installation:

```
npm install axios
```

GET request example:

```
axios.get("url")
```

POST request example:

```
axios.post("url", data)
```

27. Frontend ↔ Backend Flow

Flow:

```
Frontend
  ↓ request
Backend API
  ↓ response
Frontend updates UI
```

This is the foundation of full-stack development.

28. CORS

CORS = Cross-Origin Resource Sharing.

Frontend:

```
localhost:5173
```

Backend:

```
localhost:5000
```

Different ports = different origins.

Browser blocks communication unless backend allows it.

Installed using:

```
npm install cors
```

Used as:

```
app.use(cors())
```

29. async/await

Used for asynchronous operations.

API requests take time.

Example:

```
const response = await axios.post(...)
```

Meaning:

- wait for API response
 - then continue execution
-

30. Git

Git tracks project history locally.

Initialization:

```
git init
```

Useful for:

- checkpoints
 - version control
 - restoring older code
-

31. GitHub

GitHub stores Git repositories online.

Benefits:

- backup
 - portfolio
 - collaboration
-

32. Git Workflow

Save changes locally:

```
git add .  
git commit -m "message"
```

Push online:

```
git push
```

33. .gitignore

Used to ignore unnecessary/sensitive files.

Example:

```
node_modules  
.env  
dist
```

34. LEARNINGS.md

Personal engineering notes file.

Purpose:

- track concepts
 - revise later
 - build understanding gradually
-

35. Important Understanding From Day 1

The goal is NOT memorizing syntax.

The goal is understanding:

- how frontend works
- how backend works
- how APIs communicate
- how data flows
- how modern applications are structured

Everything else becomes easier with practice.

36. Current Project Status

Completed:

- React setup
- component basics
- props
- state
- forms
- Express backend setup
- API creation
- frontend-backend communication
- POST requests
- Axios
- CORS
- Git + GitHub setup

Next Step:

- MongoDB
- real user storage
- authentication